



CLAUDE CODE · AVANÇADO · AULA 4

Agent Teams

Orquestração paralela

Múltiplos sub-agents trabalhando ao mesmo tempo sobre um problema. O orquestrador dispara, aguarda todos e sintetiza uma resposta unificada.

Fan-Out / Fan-In

orchestrator.md

Task tool

Síntese



De uma instância a um *time*

Clique em cada etapa da Stack de Extensão.

1

Skills

2

Hooks

3

Sub-Agents

4

Teams

Skills — macros

Um Claude com instruções aprimoradas. Segue regras, mas ainda é uma única instância fazendo tudo em sequência.

Hooks — automação determinística

Scripts que interceptam ações do Claude. Ainda uma instância, mas com guardrails automáticos ao redor.

Sub-Agents — especialização

Instâncias isoladas com permissões específicas. O orquestrador delega para **um agente de cada vez** — ainda sequencial.

Agent Teams — paralelismo

Múltiplos sub-agents rodando **simultaneamente**. O orquestrador delega para vários, recebe todos e sintetiza.

O que são *Agent Teams*?

Grupos de sub-agents que trabalham em **paralelo** sobre um mesmo problema. O orquestrador dispara vários, aguarda todos e **sintetiza**.



Sub-Agent (Aula 3) — sequencial

▶ security-agent
~2 min

▶ test-agent
~2 min

▶ docs-agent
~2 min

🕒 Total: ~6 min (em fila)



Agent Team (Aula 4) — paralelo

security
~2 min

test
~2 min

docs
~2 min

⚡ síntese consolida os 3

⚡ Total: ~2 min — 3x mais rápido!

Como o orquestrador *coordena*

1

Análise da tarefa

Decompõe o problema em sub-tarefas independentes

2

Seleção dos agents

Escolhe os mais adequados pelas descriptions

3

Disparo paralelo

Invoca todos via Task tool, contexto isolado

4

Aguarda resultados

Espera o mais lento; falhas não bloqueiam

5

Síntese

Consolida, remove duplicatas, prioriza

6

Entrega

Relatório final: crítico / aviso / aprovado



"Revise este PR" → o orquestrador decompõe em **security + quality + tests + docs**, dispara os 4 em paralelo com `Promise.all([...])` e consolida em um único relatório.

Montando um *time completo*

Time de PR review com 4 agents rodando em paralelo.

```
.claude/agents/review-team/  
├── orchestrator.md  
├── security.md  
├── quality.md  
├── tests.md  
└── docs.md
```



O **orquestrador** invoca security, quality, tests e docs **em paralelo**.



security-reviewer

OWASP Top 10, secrets, CVEs · Bash(grep), Read



quality-reviewer

Legibilidade, naming, complexidade · Read



test-validator

Cobertura e qualidade · Bash(npm test), Read



docs-checker

Funções, tipos e APIs documentados · Read

O *orchestrator.md*

Define como coordenar o time, em qual ordem disparar e como consolidar.

```
---
name: pr-review-orchestrator
description: Coordena o time de PR review.
tools:
  - Task # invoca sub-agents
---

# Protocolo de execução
1. Receba o diff/PR
2. Dispare em PARALELO:
  - Task(security-reviewer, diff)
  - Task(quality-reviewer, diff)
  - Task(test-validator, diff)
3. Aguarde TODOS concluírem
4. Consolide no formato:
  ### 🚨 Crítico (bloqueia merge)
  ### ⚠️ Avisos
  ### ✅ Aprovado
```

🎯 Task tool

Como o orquestrador invoca: `Task(nome, instrução)` dispara uma instância isolada.

🚀 Disparo paralelo

Listar os Tasks juntos faz o Claude Code executá-los **simultaneamente**.

⌚ Aguarda todos

"Aguarde TODOS concluírem" impede avançar para a síntese cedo demais.

📄 Formato estruturado

Definir o formato final garante consistência entre execuções.

Padrões de *coordenação*

Clique nas abas para ver cada topologia.

Fan-Out / Fan-In

Pipeline

Revisão cruzada

Especialista dinâmico

Orch → [A1, A2, A3] → Orch (síntese)

Um orquestrador dispara N agents em paralelo e consolida N resultados em 1.

O padrão mais comum.

📌 Quando a tarefa tem N dimensões independentes (segurança, qualidade, testes...).

Orch → A1 → A2 → A3 → Orch

A1 processa e passa para A2, que passa para A3. Cada etapa enriquece o output anterior.

📌 Quando cada agent depende do anterior (lint → test → build → deploy).

Orch → A1 (cria) → A2 (revisa A1) → Orch

A faz o trabalho, B revisa A, C revisa B. Valida com múltiplas perspectivas.

📌 Quando a qualidade é crítica e não pode haver falhas (produção, finanças).

Time de *refatoração*

"Refatore o módulo de auth para Clean Architecture" — **clique** em cada fase.



Fase 1 • Análise inicial

O **orquestrador** lê o módulo `auth/`, identifica padrões a melhorar e decide quais agents convocar.

Fase 2 • Trabalho paralelo

3 agents simultâneos: **architect-agent** planeja a estrutura Clean Arch · **code-agent** refatora · **test-agent** atualiza os testes.

Fase 3 • Validação

O orquestrador aguarda os 3 terminarem e verifica se os testes passam na nova estrutura.

Fase 4 • Revisão cruzada

O **security-agent** recebe o código refatorado e checa se a refatoração não introduziu vulnerabilidades.

Fase 5 • Relatório final

Consolida: o que mudou, testes passando, vulnerabilidades verificadas. Apresenta o diff resumido.

Boas práticas & *armadilhas*

✓ Boas práticas

- ✓ Mantenha cada agent focado em **1 domínio**
- ✓ Defina o **formato de saída** (JSON estruturado)
- ✓ Escreva um `orchestrator.md` explícito
- ✓ Use **model: haiku** para agents simples
- ✓ Teste cada agent **individualmente** antes

✗ Armadilhas comuns

- ✗ Agents com **escopos sobrepostos** (custo duplo)
- ✗ Agents escrevendo no **mesmo arquivo** (conflito)
- ✗ Síntese **sem formato definido**
- ✗ Times **grandes demais** (> 5 agents)
- ✗ Ignorar o **custo de tokens** ($N \text{ agents} = N \times$)

Casos de uso *potentes*



Pipeline de PR automatizado

security + quality + tests + docs em paralelo → relatório com bloqueadores, avisos e aprovações. **~2 min vs ~8 min.**



Análise de arquitetura

arch-analyzer + perf + security + deps → mapa completo de saúde técnica do projeto numa única invocação.



Geração de feature completa

design + code + test + docs em paralelo → feature com testes e documentação em fração do tempo usual.



Auditoria de segurança

owasp + deps + secrets + config → 4 perspectivas simultâneas, relatório enterprise-grade a cada release.

⚡ Revisão 4x mais rápida, 100% consistente, sem custo humano no pré-check

Teste *rápido*

Uma tarefa tem 4 etapas onde cada uma depende do resultado da anterior (lint → test → build → deploy). Qual padrão de coordenação?

A Fan-Out / Fan-In

B Pipeline sequencial

C Revisão cruzada

D Especialista dinâmico

Resposta: B — Pipeline sequencial. Quando cada etapa **depende** da anterior, o paralelismo não se aplica: o output de um agent é a entrada do próximo (A1 → A2 → A3). Fan-Out/Fan-In serve para dimensões **independentes** que podem rodar ao mesmo tempo.



 BLOCO 1 COMPLETO · STACK DE EXTENSÃO

Muitos agentes, *um resultado*



Skills

SKILL.md, auto-invoke



Hooks

Pre/Post, Stop, HTTP



Sub-Agents

.claude/agents/, tool restrictions



Agent Teams

paralelismo, síntese

Fan-Out/Fan-In · orchestrator.md · **3x mais rápido** · Task tool. Próxima aula → **/loop**: tarefas agendadas.